Nama : Zenny Nurul Zulaikoh

NIM : 43050240056

Kelas : Teknologi Infromasi – 4C

Link Github : https://github.com/zennynurul/tugas-laravel-multipage

---

LAPORAN PRAKTIKUM UTS : IMPLEMENTASI CRUD PADA PROJEK YANG SUDAH DIBUAT

I: PENDAHULUAN

Latar belakang dari pengembangan proyek ZenBlog ini bermula dari keinginan untuk menciptakan sebuah wadah digital yang tidak hanya sekadar menjadi media penyimpanan informasi, namun juga sebagai representasi dari kejujuran intelektual seorang mahasiswa Teknologi Informasi. Di era di mana data mengalir begitu cepat, kebutuhan akan sebuah sistem yang mampu mengelola informasi secara mandiri menjadi sangat krusial. ZenBlog dirancang sebagai langkah awal untuk memahami bagaimana sebuah data dapat diciptakan, dikelola, dan dipelihara dengan struktur yang baik.

Tujuan utama dari praktik ini adalah untuk mengimplementasikan sebuah sistem manajemen konten yang bersifat dinamis. Melalui proyek ini, penulis berusaha untuk mengeksplorasi kemampuan kerangka kerja Laravel dalam menangani interaksi database yang kompleks namun tetap memberikan pengalaman pengguna yang sederhana. Selain itu, praktik ini bertujuan untuk mengasah ketajaman logika dalam menyusun alur navigasi aplikasi agar setiap proses manajemen data terasa intuitif dan efisien bagi penggunanya.

Manfaat yang diharapkan dari penyusunan proyek ini adalah penguasaan kompetensi dasar dalam pengembangan perangkat lunak, khususnya pada aspek pengelolaan data. Dengan berhasil membangun sistem ini, penulis mendapatkan pemahaman mendalam mengenai bagaimana integrasi antara logika pemrograman di sisi server dan antarmuka di sisi pengguna dapat berjalan selaras. Proyek ini juga menjadi bukti nyata dari penerapan teori akademik ke dalam solusi praktis yang dapat digunakan secara langsung.

II: LANDASAN TEORI

Dalam membangun ZenBlog, pilar utama yang digunakan adalah konsep CRUD yang merupakan singkatan dari Create, Read, Update, dan Delete. Konsep ini merupakan operasi dasar yang harus dimiliki oleh setiap aplikasi yang berorientasi pada data. Melalui Create, sistem mampu menerima masukan informasi baru dari pengguna untuk kemudian disimpan ke dalam memori permanen. Sementara itu, Read memungkinkan sistem untuk memanggil kembali informasi tersebut agar dapat dikonsumsi oleh audiens dalam berbagai format tampilan.

Operasi Update memberikan fleksibilitas bagi pengguna untuk memperbaiki atau memperbarui informasi yang sudah ada tanpa harus menghapus data lama secara keseluruhan. Hal ini sangat penting untuk menjaga aktualitas sebuah konten di dalam blog. Di sisi lain, Delete menjadi instrumen terakhir yang memberikan kendali penuh kepada pengguna untuk membersihkan data yang sudah tidak relevan lagi dari sistem. Keempat proses ini bekerja secara simultan dalam sebuah siklus hidup data yang teratur.

Teknologi pendukung yang memungkinkan hal ini terjadi adalah Laravel sebagai kerangka kerja yang menyediakan Eloquent ORM. Eloquent bertindak sebagai jembatan yang sangat manusiawi karena memungkinkan pengembang untuk berinteraksi dengan database menggunakan bahasa pemrograman yang lebih natural tanpa harus menulis kueri SQL yang rumit. Untuk penyimpanan datanya, digunakan SQLite yang merupakan sistem database berbasis file tunggal. SQLite dipilih karena kepraktisannya dalam pengembangan skala kecil hingga menengah, di mana seluruh data disimpan secara permanen di dalam satu file fisik yang tetap ada meskipun aplikasi dimatikan atau dijalankan kembali.

II: METODOLOGI DAN ANALISIS SISTEM

Metodologi yang diterapkan dalam pembangunan ZenBlog berfokus pada efisiensi pengelolaan data menggunakan pola arsitektur yang memisahkan logika bisnis dengan tampilan antarmuka. Analisis sistem dilakukan untuk memastikan bahwa setiap perintah yang diberikan pengguna dapat diproses oleh aplikasi secara akurat. Hal ini dimulai dengan mendefinisikan rute atau jalur akses yang jelas untuk setiap fitur manajemen artikel, sehingga aplikasi memahami kapan harus menampilkan data, menyimpan data baru, atau memperbarui informasi yang sudah ada.

Penulis menganalisis bahwa alur navigasi yang ideal bagi sebuah blog adalah menjaga pengguna tetap berada pada konteks tugasnya. Oleh karena itu, sistem dianalisis agar selalu mengarahkan pengguna kembali ke pusat data utama setelah melakukan modifikasi. Pada bagian ini, penulis menyiapkan struktur database yang ringan namun permanen menggunakan file fisik, sehingga semua ide yang dituangkan tidak akan hilang saat aplikasi dihentikan. Analisis juga mencakup penanganan kesalahan yang mungkin terjadi, seperti pencarian data yang tidak tersedia, agar sistem tetap stabil dan profesional saat digunakan.

IV: IMPLEMENTASI DAN PEMBAHASAN

1. **Struktur Arsitektur dan Penempatan Kode**
   Implementasi proyek ZenBlog didasarkan pada arsitektur *Model-View-Controller* (MVC) yang bertujuan untuk memisahkan tanggung jawab antara logika data, pemrosesan, dan tampilan antarmuka. Penulis menempatkan setiap komponen pada direktori standar Laravel guna memastikan kemudahan dalam pemeliharaan kode. Seluruh interaksi database dikelola melalui Model yang terhubung dengan file penyimpanan permanen SQLite, sementara alur navigasi dikendalikan sepenuhnya melalui sebuah Controller pusat.

Kode ArticleController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Article;
use Illuminate\Http\Request;

class ArticleController extends Controller
{
    public function index()
    {
        $articles = Article::latest()->get();
        return view("articles", ["articleList" => $articles]);
    }

    public function create() { return view("create_article"); }

    public function store(Request $request)
    {
        $validatedData = $request->validate([
            'title' => 'required|max:255',
            'category' => 'required',
            'image' => 'required|url',
            'description' => 'required'
        ]);

        Article::create($validatedData);
        // Kembali ke Katalog
        return redirect('/artikel')->with('success', 'Artikel berhasil
dibuat!');
    }

    public function edit($id)
    {
        $article = Article::findOrFail($id);
        return view("edit_article", compact("article"));
    }

    public function update(Request $request, $id)
    {
        $article = Article::findOrFail($id);
        $article->update($request->all());
        // Kembali ke Katalog
        return redirect('/artikel')->with('success', 'Artikel berhasil
diupdate!');
    }
```

```
    public function destroy($id)
    {
        Article::destroy($id);
        return redirect('/artikel');
    }

    public function show($id)
    {
        $article = Article::findOrFail($id);
        return view("detail_article", compact("article"));
    }
}
```

2. **Pembahasan Mendalam Logika CRUD**
   Bagian ini menguraikan bagaimana setiap operasi manajemen data diterapkan untuk menciptakan sistem yang tangguh.

   A. **Operasi Create dan Validasi**
   Proses penambahan artikel baru diawali dengan fungsi validasi yang ketat. Sistem memastikan bahwa setiap masukan pengguna, mulai dari judul hingga alamat URL gambar, telah memenuhi kriteria yang ditetapkan sebelum instruksi penyimpanan dijalankan. Hal ini krusial untuk mencegah terjadinya kegagalan sistem akibat data yang tidak lengkap atau tidak valid.

   Kode fungsi store()

```
public function store(Request $request)
    {
        $validatedData = $request->validate([
            'title' => 'required|max:255',
            'category' => 'required',
            'image' => 'required|url',
            'description' => 'required'
        ]);

        Article::create($validatedData);
        // Kembali ke Katalog
        return redirect('/artikel')->with('success', 'Artikel berhasil
dibuat!');
    }
```

Penjelasan*:* Pada kode di atas, instruksi redirect('/artikel') adalah kunci utama yang memastikan bahwa setelah artikel berhasil dipublikasikan, sistem secara otomatis mengalihkan pengguna kembali ke halaman katalog untuk melihat hasilnya secara langsung.

B. Operasi Read dan Pencarian Data

Sistem menyediakan dua cara dalam menyajikan informasi. Pertama, melalui fungsi indeks yang menampilkan daftar ringkas seluruh artikel, dan kedua melalui fungsi detail yang mencari data berdasarkan identitas unik (ID) artikel tersebut untuk ditampilkan secara utuh.

Kode fungsi index() dan

```
    public function index()
    {
        $articles = Article::latest()->get();
        return view("articles", ["articleList" => $articles]);
    }
```

Kode show()

```
    public function show($id)
    {
        $article = Article::findOrFail($id);
        return view("detail_article", compact("article"));
    }
```

Penjelasan: Penggunaan metode findOrFail($id) memberikan keamanan tambahan di mana sistem akan secara otomatis memberikan respon "Not Found" jika pengguna mencoba mengakses artikel dengan ID yang tidak terdaftar, sehingga aplikasi tidak mengalami *crash*.

C. Operasi Update dan Delete

Pembaruan data dilakukan dengan memanggil data lama ke dalam formulir, membiarkan pengguna melakukan modifikasi, dan menimpanya dengan nilai baru. Sementara itu, penghapusan data bertugas untuk membersihkan baris informasi yang sudah tidak relevan lagi secara permanen dari database.

Kode fungsi update() dan

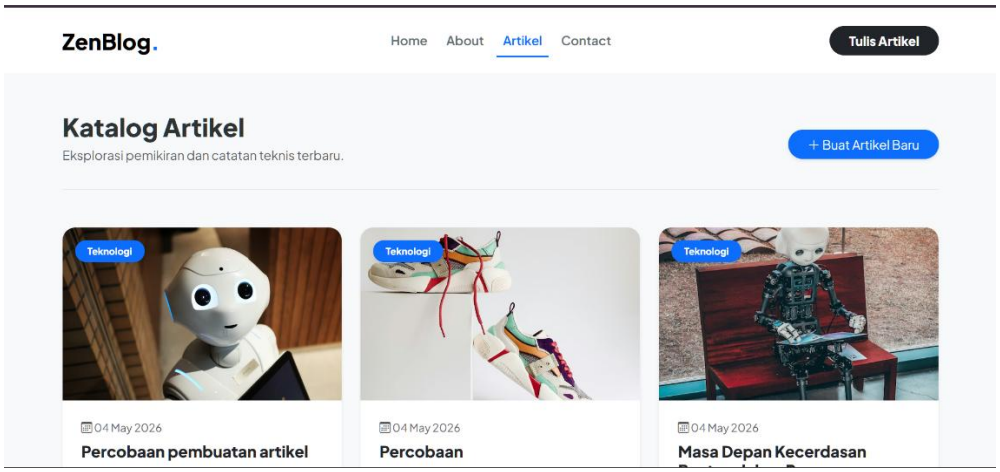
```
    public function update(Request $request, $id)
    {
        $article = Article::findOrFail($id);
        $article->update($request->all());
        // Kembali ke Katalog
        return redirect('/artikel')->with('success', 'Artikel berhasil
diupdate!');
    }
```

Kode destroy()

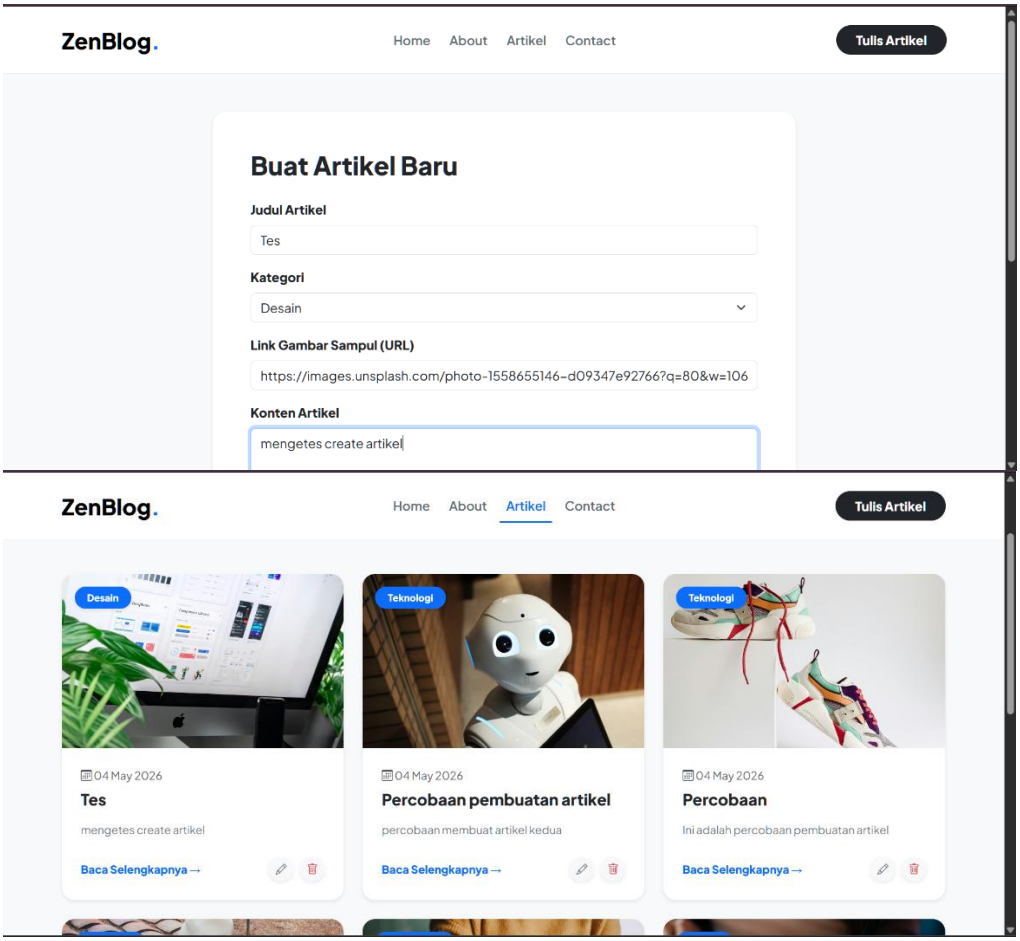
```
    public function destroy($id)
    {
        Article::destroy($id);
        return redirect('/artikel');
    }
```

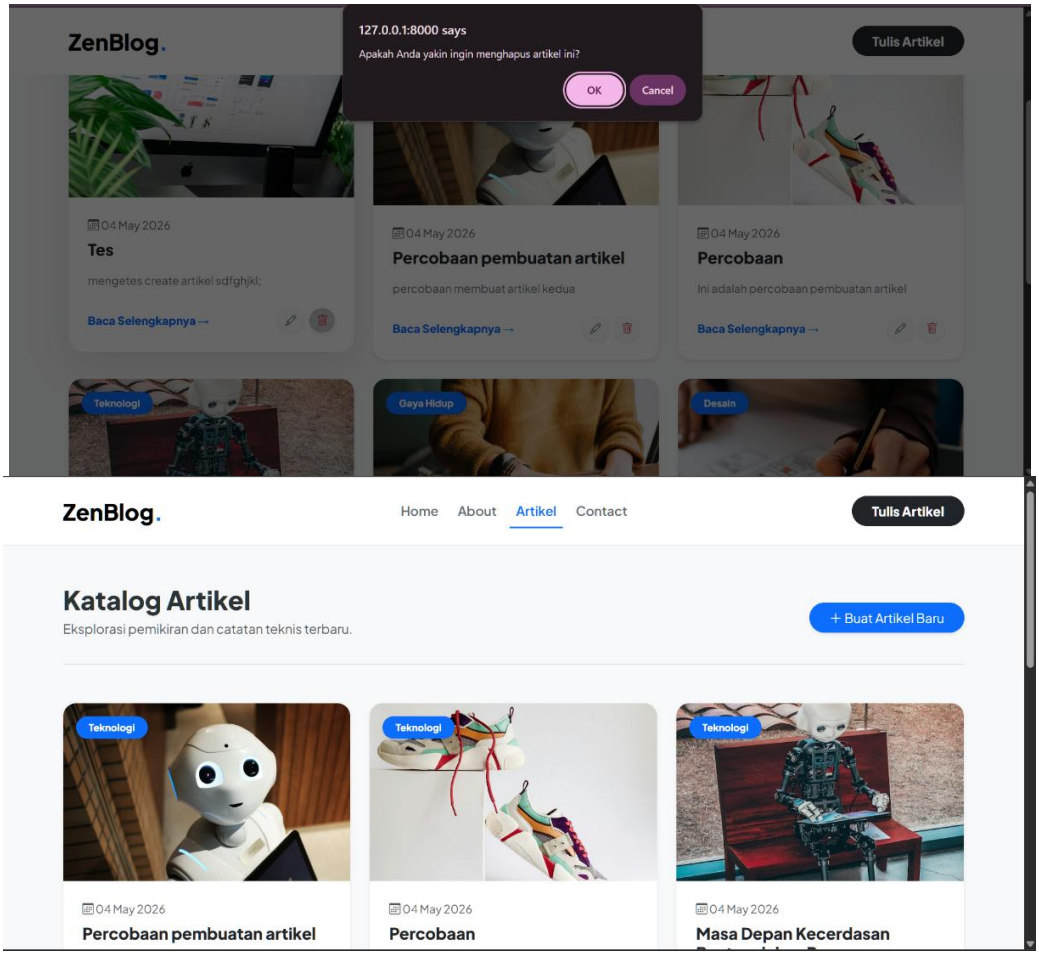
3. **Analisis Antarmuka dan Output Sistem**

Tahap akhir dari implementasi ini adalah menyajikan data mentah dari database ke dalam tampilan visual yang dapat dinikmati oleh pembaca. Penulis menggunakan mesin template Blade untuk memetakan setiap properti data ke dalam komponen UI Bootstrap yang responsif. Analisis terhadap hasil keluaran menunjukkan bahwa integrasi antara logika di sisi server dan tampilan di sisi pengguna telah berjalan selaras.

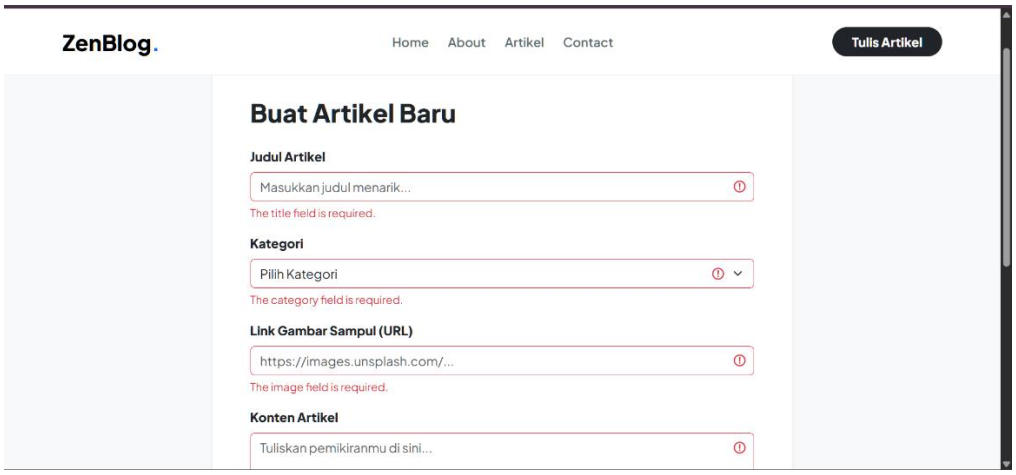


(Halaman Katalog Artikel /artikel) Analisis: Gambar di atas menunjukkan keberhasilan fungsi **Read**, di mana data dari database SQLite ditampilkan dalam bentuk kartu informasi yang tertata rapi.

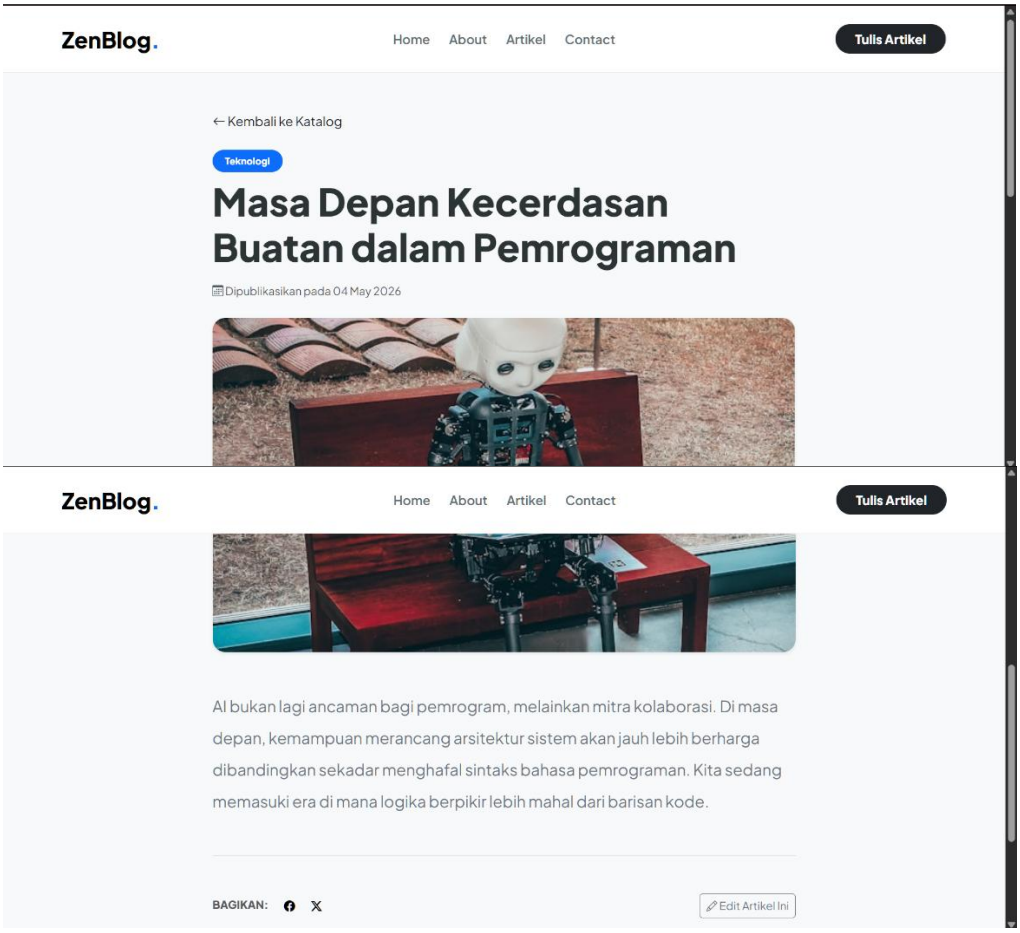




(Halaman Create Artikel dan Pesan Delete) Analisis: Tampilan ini merepresentasikan fitur konfirmasi pada operasi **Delete**. Sistem memberikan jendela dialog konfirmasi untuk memastikan aspek keamanan data sebelum penghapusan dilakukan. Berbeda dengan operasi **Update** yang bersifat instruksi langsung, penghapusan ini memerlukan validasi manual demi mencegah kesalahan fatal.



(Pesan Validasi saat input kosong) Analisis: Output ini membuktikan bahwa sistem keamanan data (validasi) bekerja dengan baik dalam mencegah masuknya informasi yang tidak lengkap ke dalam database.



(Halaman Detail Artikel saat diklik) Analisis: Tampilan detail ini menunjukkan kemampuan sistem dalam memproses parameter id secara dinamis untuk menyajikan konten artikel secara mendalam dan fokus.

BAB V: PENUTUP

Kesimpulan

Berdasarkan seluruh rangkaian proses pengembangan yang telah dilakukan, dapat disimpulkan bahwa implementasi sistem CRUD pada aplikasi ZenBlog telah berhasil berjalan dengan optimal. Proyek ini membuktikan bahwa penggunaan kerangka kerja Laravel mampu menyederhanakan interaksi kompleks dengan database melalui penggunaan Eloquent ORM, sehingga pengelolaan artikel dapat dilakukan secara terstruktur. Pemanfaatan SQLite sebagai sistem database berbasis file terbukti efektif dalam menjaga persistensi data, di mana informasi tetap tersimpan secara permanen meskipun aplikasi dinonaktifkan.

Keberhasilan utama dalam praktikum ini juga terletak pada pengaturan alur navigasi yang intuitif. Dengan memastikan setiap fungsi manajemen—baik itu saat menambah, mengedit, maupun membatalkan aksi—selalu mengarahkan pengguna kembali ke halaman katalog artikel, aplikasi memberikan pengalaman penggunaan yang lebih profesional dan fokus. Hal ini meminimalkan disorientasi pengguna dan memastikan bahwa setiap perubahan konten dapat segera diverifikasi pada halaman penyajian informasi yang tepat. Secara keseluruhan, ZenBlog telah memenuhi standar aplikasi manajemen konten dasar yang tangguh, fungsional, dan ramah bagi pengguna.